

Test-Driven Development, Mocking, and Dependency Isolation

SE 413/513 — Software Engineering

Course Reading Material

Unit: Test-Driven Development, Test Doubles, and Testable Design

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,  
author = {Hasan, Kamrul},  
year = {2026},  
title = {Software Testing},  
institution = {Grand Valley State University},  
url = {https://github.com/mdkamrulhasan/software-testing}  
}
```

Contents

1	Learning Objectives	3
2	What Is Test-Driven Development?	3
2.1	The Red-Green-Refactor Cycle	3
2.2	A Worked Example: Building a Password Validator	4
3	Why TDD Forces Testable Design	5
4	Dependency Isolation and Seams	5
4.1	Dependency Injection	6
5	The Test Double Taxonomy	6
5.1	Dummy	7
5.2	Stub	8
5.3	Fake	8
5.4	Spy	8
5.5	Mock	9
5.6	Summary of the Taxonomy	9
6	State Verification vs. Behavior Verification	10
6.1	Trade-offs	10
7	Common Pitfalls	11

8 A Practical Mental Model	11
9 Summary	12
10 Practice Exercises	12
11 Discussion Questions	13
12 Key Takeaways for SE 413/513	14
13 References	14

1 Learning Objectives

After completing this reading, you should be able to:

- Explain the Red-Green-Refactor cycle and apply it to build a function incrementally.
- Explain why writing tests first tends to produce more testable designs.
- Identify hard-coded dependencies and refactor them using dependency injection.
- Define and distinguish the five kinds of test double: dummy, stub, fake, spy, and mock.
- Explain the difference between state verification and behavior verification.
- Contrast the “classical” (Detroit) and “mockist” (London) schools of TDD.
- Recognize the risks of over-mocking and mocks that do not match real dependency behavior.

2 What Is Test-Driven Development?

Test-driven development (TDD) is a software development practice in which you write a failing test *before* you write the production code that makes it pass.[3, 1] Rather than writing code and then (maybe) testing it afterward, TDD inverts the order: the test comes first, and it drives the design of the code that follows.

Real-World Case Study

The Origins of TDD. Kent Beck popularized test-driven development as part of **Extreme Programming (XP)** in his 1999 book *Extreme Programming Explained: Embrace Change*, and later devoted an entire book to the practice, *Test-Driven Development: By Example* (2002).[2, 3] Beck himself describes TDD not as something he invented outright, but as a rediscovery of a discipline he had seen used informally decades earlier. Whatever its precise origin, Beck’s books are what turned it into a widely taught, named practice.

2.1 The Red-Green-Refactor Cycle

TDD is organized around a short, repeating cycle:

1. **Red:** Write a small test for a piece of behavior that does not exist yet. Run it, and watch it fail. This step matters more than it sounds: a test you have never seen fail is a test you cannot fully trust to catch a real defect.
2. **Green:** Write the *minimum* code necessary to make the test pass — no more. It is fine, even expected, for this code to be incomplete or overly simple at this stage.
3. **Refactor:** With the safety net of a passing test suite, clean up the code (and the test, if needed) without changing its behavior. Remove duplication, improve names, simplify structure. Re-run the tests after every change to confirm they stay green.

Key Idea

The discipline of writing only the *minimum* code to pass the current test, resisting the urge to build out the “obviously needed” functionality all at once, is what keeps each cycle short and each test meaningfully connected to a specific piece of behavior. Kent Beck calls the deliberate use of a deliberately oversimplified implementation “**fake it till you make it**”[3]: write the simplest possible thing that passes (even a hard-coded constant), and let the *next* test force you to generalize.

2.2 A Worked Example: Building a Password Validator

Suppose we need a function `is_strong_password(password)` that returns `True` only for passwords that are at least 8 characters long, contain at least one digit, and contain at least one uppercase letter. Rather than writing the whole function at once, TDD builds it up one small cycle at a time.

Cycle 1. Red: write a test for a password we know should be rejected.

```
def test_rejects_short_password():
    assert is_strong_password("Ab1") is False
```

Green: the simplest possible implementation that passes — “fake it”:

```
def is_strong_password(password):
    return False
```

This is obviously not a complete implementation, but it is honest: nothing yet requires it to be anything more.

Cycle 2. Red: write a test for a password we know should be accepted, forcing the fake implementation to break.

```
def test_accepts_long_password_with_digit_and_uppercase():
    assert is_strong_password("Abcdefg1") is True
```

Green: this test now fails against `return False`, forcing real logic:

```
def is_strong_password(password):
    if len(password) < 8:
        return False
    return True
```

Cycle 3. Red: this passes both prior tests, but nothing yet checks the digit requirement.

```
def test_rejects_password_without_digit():
    assert is_strong_password("Abcdefgh") is False
```

Green:

```
def is_strong_password(password):
    if len(password) < 8:
        return False
    if not any(char.isdigit() for char in password):
        return False
    return True
```

Cycle 4. Red: similarly, nothing yet checks the uppercase requirement.

```
def test_rejects_password_without_uppercase():
    assert is_strong_password("abcdefg1") is False
```

Green:

```
def is_strong_password(password):
    if len(password) < 8:
        return False
    if not any(char.isdigit() for char in password):
        return False
    if not any(char.isupper() for char in password):
        return False
    return True
```

Refactor. All four tests pass. Now, and only now, clean up the implementation:

```
def is_strong_password(password):
    rules_satisfied = [
        len(password) >= 8,
        any(char.isdigit() for char in password),
        any(char.isupper() for char in password),
    ]
    return all(rules_satisfied)
```

Re-running all four tests after the refactor confirms nothing broke. Notice that the four tests written along the way are not thrown away — they remain as a small regression suite, a natural side effect of developing this way.

Tip

Each of the four tests above corresponds naturally to an equivalence class from the *Equivalence Partitioning* reading: too-short passwords, passwords missing a digit, and passwords missing an uppercase letter are each their own invalid class. TDD and systematic test-case design are complementary; TDD gives you a *process* for growing code one test at a time, while equivalence partitioning and boundary-value analysis help you choose *which* tests to write at each step.

3 Why TDD Forces Testable Design

Writing the test first has a side effect that turns out to be at least as valuable as the tests themselves: **it is very hard to write a test-first test for code that is badly designed for testing.** If a piece of logic reaches out to a real database, makes a real network call, or reads the real system clock in the middle of a function, writing a fast, repeatable test for it *before* the code exists forces you to confront that problem immediately, rather than discovering it later when a teammate tries (and struggles) to test your finished code.

Key Idea

A function or class that is hard to unit test is usually not “fine, just annoying to test” — it is very often a symptom of a real design problem: hidden dependencies, mixed responsibilities, or global/shared state. TDD’s real long-term value is arguably less about the tests themselves and more about the design pressure that writing them first applies.

4 Dependency Isolation and Seams

Consider a service that emails a user a password-reset code:

```
import smtplib
```

```
class PasswordResetService:
    def send_reset_email(self, user_email, reset_token):
        message = f"Subject: Password Reset\n\nYour reset code is {reset_token}"
        server = smtplib.SMTP("smtp.example.com")
        server.sendmail("noreply@example.com", user_email, message)
        server.quit()
```

This class has a **hard-coded dependency**: it constructs its own connection to a real SMTP server, deep inside the method we want to test. Testing `send_reset_email()` as written would require a real (or realistic fake) mail server just to run — slow, flaky, and entirely unnecessary for checking that the service builds the right message and asks the mail system to send it to the right address.

4.1 Dependency Injection

Dependency injection solves this by having the class *accept* its dependency from the outside, instead of constructing it internally:

```
class PasswordResetService:
    def __init__(self, email_client, logger):
        self.email_client = email_client
        self.logger = logger

    def send_reset_email(self, user_email, reset_token):
        message = f"Subject: Password Reset\n\nYour reset code is {reset_token}"
        self.email_client.send(user_email, message)
```

In production, `PasswordResetService` is constructed with a real `SmtplibEmailClient` that wraps `smtplib`. In a test, it can be constructed with any object that has a `.send(to, message)` method — no real network required. This is called **constructor injection**, the most common form of dependency injection: dependencies arrive as constructor parameters rather than being created inside the class.

Key Idea

Michael Feathers, in *Working Effectively with Legacy Code*, calls a place where you can alter a program's behavior without editing the code in that exact spot a **seam**. The `email_client` parameter above is a seam: production code and test code can each supply a different implementation through the exact same seam, without the `PasswordResetService` class itself ever changing.

Tip

Common hard-coded dependencies worth watching for: direct instantiation of a database connection, an HTTP client, a file handle, `datetime.now()` (the real system clock), or `random` (real randomness). Each one makes a unit test slower, less deterministic, or dependent on an external system — exactly the properties the *Testing Foundations* reading identified as reasons to reach for a test double in the first place.

5 The Test Double Taxonomy

The general term **test double** refers to any stand-in object used in place of a real dependency. In practice, test doubles come in several distinct flavors, each suited to a different testing job. The most widely used vocabulary for these flavors comes from Gerard Meszaros's book *xUnit Test Patterns: Refactoring Test Code*.

(2007), which coined “test double” itself (by analogy to a film stunt double) and formalized five categories: **dummy**, **stub**, **fake**, **spy**, and **mock**.

We will illustrate all five using the `PasswordResetService` from Section 4.

5.1 Dummy

A **dummy** is a simple object, value, or component that is used only to satisfy a requirement of the system under test, without actually being used in the test. Think of it as a placeholder.

A simple example. Suppose you have:

```
def calculate_salary(employee, database):
    salary = database.get_salary(employee)
    return salary * 1.10
```

To test `calculate_salary()`, you need to provide both an `employee` and a `database`. If the test does not actually need employee information, you can use a dummy employee:

```
dummy_employee = "John"
```

The test can then focus on the salary calculation itself, rather than on the employee object — `dummy_employee` could just as easily be `None` or an empty string; `calculate_salary()` never looks inside it.

Tip

A useful analogy. Imagine a restaurant reservation system requires a phone number, but your test only exercises the table-allocation algorithm. You might supply `phone = "000-000-0000"`: the algorithm doesn’t care about the phone number, so it is a **dummy value** — present only because the function requires it. In short: a dummy is something supplied because it is required, but whose actual value is irrelevant to what the test is checking.

The same idea applies back in the `PasswordResetService` world, to the constructor’s `logger` parameter whenever a test’s code path never calls it:

```
def test_reset_email_is_sent_to_the_right_address():
    dummy_logger = object() # required by the constructor, never used here
    service = PasswordResetService(FakeEmailClient(), dummy_logger)
    service.send_reset_email("alice@example.com", "123456")
    # ...assertions about the email client go here; dummy_logger is untouched
```

Tip

A dummy can also be **poisoned** on purpose, so that any accidental use fails loudly and clearly instead of with a generic error:

```
class DummyLogger:
    def log(self, message):
        raise AssertionError("DummyLogger should never be called in this test")
```

Either version is a dummy — the defining trait is that a *correct* test never actually calls it. Poisoning just turns an unexpected use into an immediate, unmistakable test failure with a clear message.

5.2 Stub

A **stub** provides a canned answer to a call, with no expectations about how it is used — the test does not check whether or how the stub was called.

```
class StubEmailClient:
    """A stub: always gives the same canned answer, so the
    code under test has something to work with."""
    def send(self, to, message):
        return True

def test_send_reset_email_completes_without_error():
    service = PasswordResetService(StubEmailClient(), logger=DummyLogger())
    service.send_reset_email("alice@example.com", "123456") # runs to completion
```

The stub simply lets `send_reset_email()` run without touching a real mail server. Notice what the test does *not* do: it never inspects `StubEmailClient` afterward, and it never asserts that `send()` was called. That is exactly what separates a stub from the fake and spy we meet next.

5.3 Fake

A **fake** is a lightweight but *genuinely working* implementation — good enough for tests, but not suitable for production (perhaps because it is not persistent, not secure, or not scalable).

```
class FakeEmailClient:
    """A fake: a real, working implementation good enough for tests
    (an in-memory 'sent mail' list), but not fit for production."""
    def __init__(self):
        self.sent_emails = []

    def send(self, to, message):
        self.sent_emails.append((to, message))
        return True
```

A test using this fake performs **state verification**: it checks the resulting state of the fake, after the action, rather than asserting anything about the call itself.

```
def test_reset_email_is_recorded_by_the_fake():
    email_client = FakeEmailClient()
    service = PasswordResetService(email_client, logger=DummyLogger())
    service.send_reset_email("alice@example.com", "123456")

    assert len(email_client.sent_emails) == 1
    to, message = email_client.sent_emails[0]
    assert to == "alice@example.com"
    assert "123456" in message
```

5.4 Spy

A **spy** records information about how it was called, so the test can inspect that record afterward.

```
class SpyEmailClient:
    """A spy: records every call so the test can inspect it later."""
    def __init__(self):
        self.calls = []
```

```
def send(self, to, message):
    self.calls.append((to, message))
    return True
```

```
def test_email_client_is_called_exactly_once():
    spy = SpyEmailClient()
    service = PasswordResetService(spy, logger=DummyLogger())
    service.send_reset_email("alice@example.com", "123456")

    assert len(spy.calls) == 1
```

Common Pitfall

In practice, the line between a fake and a spy blurs constantly — the two code samples above are nearly identical. The distinction is mostly about *intent*: a fake aims to behave like a believable, reusable version of the real dependency (an in-memory mailbox); a spy exists purely to give a specific test something to inspect, and would not make sense as a stand-in anywhere else. Do not lose sleep over classifying a given test double perfectly — the vocabulary is most useful for communicating *purpose* to teammates, not for rigid categorization.

5.5 Mock

A **mock** is pre-configured with expectations about how it should be called, and the test verifies those expectations directly against the mock — this is **behavior verification**, checking the interaction itself rather than any resulting state.

```
from unittest.mock import MagicMock

def test_email_client_called_with_correct_message():
    mock_client = MagicMock()
    service = PasswordResetService(mock_client, logger=DummyLogger())
    service.send_reset_email("alice@example.com", "123456")

    mock_client.send.assert_called_once_with(
        "alice@example.com",
        "Subject: Password Reset\n\nYour reset code is 123456",
    )
```

5.6 Summary of the Taxonomy

Double	Purpose	Verifies via	Seen above as
Dummy	Fills a required parameter; never actually used	(nothing)	dummy_employee / DummyLogger
Stub	Returns a canned answer so the code under test can run	(nothing)	StubEmailClient
Fake	A lightweight, working implementation	State	sent_emails list
Spy	Records calls for the test to inspect afterward	State (of the spy)	calls list
Mock	Pre-configured with expectations, checked against the call itself	Behavior	assert_called_once_with()

6 State Verification vs. Behavior Verification

The dummy/stub/fake side of the taxonomy and the spy/mock side reflect two different underlying philosophies about *what a unit test should check*, often called the **classical (Detroit)** and **mockist (London)** schools of TDD.

- **Classical / Detroit school:** Prefer real objects wherever practical, and use test doubles sparingly — mainly for dependencies that are slow, external, or nondeterministic. Tests primarily check the *resulting state* of the system: what value was returned, what the object’s fields now contain.
- **Mockist / London school:** Isolate the unit under test from *every* collaborator using test doubles (typically mocks), so a test failure always points at exactly one unit. Tests primarily check *interactions*: which methods were called, with what arguments, how many times.

Real-World Case Study

Two Schools, Two Books. The mockist/London approach traces back to the paper that introduced mock objects into unit testing: “Endo-Testing: Unit Testing with Mock Objects,” by Tim Mackinnon, Steve Freeman, and Philip Craig, presented at the XP2000 conference.[7] Freeman later co-authored, with Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (2009)[8] — widely regarded as the canonical text for mockist/London-school TDD, in which nearly every collaborator is mocked and object interactions are designed and tested deliberately from the outside in. Kent Beck’s own *Test-Driven Development: By Example* (2002) leans classical[3]: prefer real collaborators, and reach for test doubles more selectively.

6.1 Trade-offs

Neither school is simply “correct.” Each carries real trade-offs:

- **Mockist tests** isolate failures precisely (a broken unit fails its own tests, not a chain of dependents’ tests too) and run fast, since nothing real is instantiated. But over-specified interaction assertions couple tests tightly to *how* code is implemented, not just *what* it does — a refactor that preserves behavior can still break many mock-based tests.
- **Classical tests** tend to survive refactoring better, since they check outcomes rather than internal call patterns, and they give more confidence that components genuinely work together. But they can be slower, and when a failure does occur deep in a chain of real collaborators, it can take more digging to isolate which component is actually at fault.

Tip

Most real projects land somewhere between the two schools: use real objects and state verification for simple, fast, deterministic collaborators; reach for stubs, fakes, spies, or mocks specifically at the boundaries where a dependency is slow, external, non-deterministic, or expensive — exactly the criteria the *Testing Foundations* reading gave for when mocking is worthwhile in the first place.[1]

7 Common Pitfalls

Common Pitfall

Skipping the Red step. If you write the implementation first and the test afterward, you never actually see the test fail — so you cannot be sure the test is capable of failing at all (a broken assertion, a typo comparing a variable to itself, or a tautological check can all pass “by accident”). Always confirm a new test fails for the expected reason before making it pass.

Common Pitfall

Mocking everything, out of habit. Not every collaborator needs to be a mock. A simple, fast, deterministic value object (a `Money` class, a data class holding validated fields) is usually better used for real in a test than replaced with a test double — mocking it adds ceremony without reducing risk.

Common Pitfall

Mocks that lie. A stub or mock is only useful if it behaves the way the real dependency actually behaves. If the real `email_client.send()` raises an exception on failure but your stub always returns `True`, your tests can pass while the real integration is broken. Constraining a double to the real class’s interface with `spec` or `autospec` (from Python’s `unittest.mock`[\[9\]](#)) catches some of these mismatches (a typo’d method name, a wrong number of arguments), though it cannot catch every behavioral mismatch on its own.

8 A Practical Mental Model

When you are about to write production code, think:

“What is the smallest failing test I could write first, and what is the smallest change that would make it pass?”

When a class is hard to instantiate or test, think:

“What hard-coded dependency is hiding in here, and what seam could I introduce to inject it instead?”

When you reach for a test double, think:

“Do I need this object to just return something (stub), actually behave like a lightweight version of the real thing (fake), let me inspect what happened afterward (spy), or actively verify how it was called (mock)?”

When a mock-heavy test breaks after a refactor that didn’t change behavior, think:

“Was this assertion checking *what* the code does, or *how* it does it internally?”

9 Summary

Term	Meaning
Test-driven development (TDD)	Writing a failing test before the code that makes it pass
Red-Green-Refactor	Fail, pass minimally, then clean up without changing behavior
Fake it till you make it	Passing a test with a deliberately oversimplified implementation, generalized later
Dependency injection	Supplying a class's dependencies from outside instead of constructing them internally
Seam	A place where behavior can be altered without editing the code at that spot
Dummy	A double passed only to satisfy a parameter; never used
Stub	A double that returns a canned answer; not itself verified
Fake	A lightweight, working implementation, verified via its resulting state
Spy	A double that records calls for the test to inspect afterward
Mock	A double whose expected interactions are asserted directly (behavior verification)
Classical / Mockist TDD	Prefer real objects and state checks / isolate everything and check interactions

Key Idea

Let the test come first, and let the design follow. TDD is as much a design discipline as a testing technique: writing the test before the code surfaces hidden dependencies early, and dependency injection turns those dependencies into seams you control. The dummy/stub/fake/spy/mock vocabulary then gives you a precise way to choose — and to communicate to your teammates — exactly what kind of test double a given test actually needs.

10 Practice Exercises

Exercise 1 — Red-Green-Refactor

Using strict TDD (one small failing test at a time), build a function `calculate_tip(bill_total, party_size)` that returns a 20% tip, except it returns a 22% tip when `party_size` is 6 or greater (a common “large party” policy). Write out each Red step and each Green step separately, in order.

Exercise 2 — Finding the Hard-Coded Dependency

The following class is difficult to unit test. Identify the hard-coded dependency, and rewrite it using constructor injection.

```
import time

class SessionManager:
    def create_session(self, user_id):
        expires_at = time.time() + 3600 # expires in one hour
        return {"user_id": user_id, "expires_at": expires_at}
```

Exercise 3 — Choosing the Right Double

For each scenario, name the most appropriate kind of test double (dummy, stub, fake, spy, or mock) and justify your choice:

1. A test needs an `InventoryRepository` to return a fixed stock count so a discount calculation can be tested.
2. A test must confirm that `send_welcome_email()` was called with exactly the new user's email address.
3. A constructor requires an `AuditLogger` parameter that the specific method under test never calls.
4. A test suite needs a realistic, in-memory key-value store to replace a slow, real cache across dozens of tests.

Exercise 4 — State vs. Behavior Verification

Rewrite the following mock-based (behavior-verification) test as a state-verification test using a fake, and explain one advantage and one disadvantage of each version.

```
def test_add_item_calls_cart_update(mock_cart):
    add_item(mock_cart, "widget", price=9.99)
    mock_cart.update.assert_called_once_with("widget", 9.99)
```

Exercise 5 — Design Question

A teammate on a mockist-leaning team says: “Every collaborator should be mocked, so a test only ever fails because of the unit it’s actually testing.” Using at least one concept from this reading, write a short argument identifying a real cost of following this rule absolutely.

11 Discussion Questions

1. Why does seeing a test fail (the Red step) matter, beyond just confirming the feature isn’t built yet?
2. In what sense does TDD function as a design tool, not just a testing tool?
3. What is a “seam,” and how does dependency injection create one?
4. Why might a poisoned dummy (one that raises an error if used) be more useful than one that silently does nothing?
5. In your own words, what is the difference between a fake and a spy? Why do practitioners disagree about where the line falls?
6. Why is a mock’s assertion described as “behavior verification” rather than “state verification”?
7. What specific risk does “mocks that lie” describe, and how do `spec/autospec` partially address it?
8. How would a classical (Detroit-school) developer and a mockist (London-school) developer likely test the same class differently?
9. Why can heavily mocked tests be more likely to break during a refactor that doesn’t change any externally visible behavior?
10. How do TDD, dependency injection, and the test double taxonomy relate to the “testing is context-dependent” principle from the *Testing Foundations* reading?

12 Key Takeaways for SE 413/513

1. **Red before Green.** A test you have never watched fail is a test you cannot fully trust.
2. **Write the minimum code to pass, then refactor.** Resist building more than the current test requires.
3. **Hard-to-test code is usually a design smell.** Hidden dependencies are the most common cause.
4. **Dependency injection creates seams.** Supply collaborators from outside; do not construct them internally.
5. **Five doubles, five purposes.** Dummy, stub, fake, spy, and mock each solve a different testing problem — know which one you actually need.
6. **State verification and behavior verification are both legitimate.** Neither the classical nor the mockist school is simply right; the best choice depends on the collaborator and the risk involved.
7. **A double is only as useful as it is honest.** A test double that does not behave like the real dependency it replaces can let a broken integration pass every test.

13 References

- Foundational TDD and Extreme Programming references.[2, 3]
- Dependency injection and seams for testable design.[4, 5]
- Test doubles and interaction-based testing.[6, 7, 8]
- Python mock and autospec documentation.[9]
- Course material on software testing principles and related testing practices.[1]

References

- [1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. <https://github.com/mdkamrulhasan/software-testing>
- [2] Beck, K. (1999). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- [3] Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.
- [4] Fowler, M. (2004). *Inversion of Control Containers and the Dependency Injection pattern*. martin-fowler.com. <https://martinfowler.com/articles/injection.html>
- [5] Feathers, M. C. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- [6] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.
- [7] Mackinnon, T., Freeman, S., & Craig, P. (2000). Endo-testing: Unit testing with mock objects. In *Extreme Programming and Flexible Processes in Software Engineering*.
- [8] Freeman, S., & Pryce, N. (2009). *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley Professional.
- [9] Python Software Foundation. (2026). *unittest.mock — mock object library*. <https://docs.python.org/3/library/unittest.mock.html>